

Al-Aqsa University

Faculty of Computers and IT

Engineering and Networks Dept.



جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم الهندسة والشبكات

Operating Systems for Intelligent Systems (ENGS2306)

Lecture 6: Bash Scripting Basics

Lecturer: Eng. Mohammed E. Alhams

Semester: Second Semester 2024/2025

Lecture 6: Bash Scripting Basics

Today's Objectives:

1. Understand the structure of a basic Bash script.
 2. Learn to use variables to store and retrieve data.
 3. Implement decision-making with if-then-else statements.
 4. Automate repetitive tasks using for and while loops.
 5. Write a simple script to automate a system task.
-

Recap: Why Do We Need Scripting?

In the last four weeks, we learned many powerful commands:

- ls, mkdir, chmod, chown (File management)
- useradd, usermod, groupadd (User management)
- ps, kill, nice (Process management)

The Problem: Manually typing these commands one by one is slow, repetitive, and prone to typos.

The Solution: Automation!

A **script** is a simple text file containing a sequence of commands. The shell executes these commands in order, turning a multi-step manual task into a single, repeatable action. This is the foundation of automation in Linux.

Anatomy of a Bash Script

A Bash script is a plain text file with three key features:

1. **The Shebang:** `#!/bin/bash`
 - This **must** be the very first line of your script.
 - It tells the operating system to use the Bash interpreter to run the commands in this file.
2. **Comments:** `# This is a comment`
 - Lines starting with `#` are ignored by the interpreter.
 - Use comments to explain what your script does and how it works. Good commenting is crucial for maintainable code.
3. **Executable Permission:**
 - By default, a new text file cannot be run as a program. You must make it executable first.

- `chmod +x my_script.sh`

To run the script: `./my_script.sh` (The `./` tells the shell to look in the current directory).

Using Variables to Store Data

Variables are named containers that hold information.

Declaring a Variable:

`VARIABLE_NAME="some value"`

- **Convention:** Use uppercase for environment variables, lowercase for script-local variables.
- **No spaces** around the `=` sign. This is a common mistake!

Accessing a Variable:

Use the `$` sign before the variable name.

`echo $VARIABLE_NAME`

- **Good Practice:** Enclose the variable in double quotes to handle spaces correctly: `echo "$VARIABLE_NAME"`

Example Script (greeting.sh):

```
#!/bin/bash
# A simple script that uses a variable.

GREETING="Hello, World!"
echo "$GREETING"
```

Getting Input: read and Command Substitution

Your scripts can be interactive or use the output of other commands.

1. Reading User Input: read

The `read` command pauses the script and waits for the user to type something.

```
#!/bin/bash
read -p "Please enter your name: " user_name
echo "Hello, $user_name! Welcome to scripting."
```

2. Command Substitution: `$(...)`

This powerful feature lets you save the output of a command into a variable.

```
#!/bin/bash
# Store the current date and hostname in variables
current_date=$(date)
my_hostname=$(hostname)

echo "The report for $my_hostname was generated on $current_date"
```

For Intelligent Systems: `cpu_temp=$(cat /sys/class/thermal/thermal_zone0/temp)`

Making Decisions: if-then-else

Conditional statements allow your script to perform different actions based on a condition.

Syntax:

```
if [ condition ]; then
    # block of code to run if condition is true
elif [ another_condition ]; then
    # block of code to run if another_condition is true
else
    # block of code to run if all conditions are false
fi
```

- **Spacing is critical!** There must be spaces inside the brackets [...].
- The [] is an alias for the test command.

Common Conditions:

- **String Comparison:** if ["\$USER" == "root"]
- **Numeric Comparison:** if [\$count -gt 10] (-gt > greater than, -lt < less than, -eq = equal to)
- **File Check:** if [-f "/etc/passwd"] (-f = is a file, -d = is a directory)

Repeating Tasks: The for Loop

A for loop is used to execute a block of code a specific number of times, iterating over a list of items.

Syntax:

```
for item in item1 item2 item3; do
    # code to execute for each item
    echo "Processing $item"
done
```

Practical Use Case (Intelligent Systems):

Check the status of multiple IoT devices on a network.

```
#!/bin/bash
# Ping a list of IoT devices

DEVICES="192.168.1.101 192.168.1.102 192.168.1.103"

for device in $DEVICES; do
    ping -c 1 $device
done
```

This script will ping each IP address one by one.

Repeating Tasks: The while Loop

A while loop continues to execute a block of code as long as a condition remains true.

Syntax:

```
while [ condition ]; do
    # code to execute while condition is true
done
```

Practical Use Case (Intelligent Systems):

A monitoring script that checks CPU temperature every 10 seconds.

```
#!/bin/bash
# Monitor CPU temperature indefinitely

while true; do
    temp=$(cat /sys/class/thermal/thermal_zone0/temp)
    # The value is in millidegrees, so divide by 1000
    echo "Current CPU temperature is: $((temp / 1000))°C"
    sleep 10
done
```

This is an infinite loop; you would stop it with Ctrl+C.

Practical Scenario: Simple System Backup Script

Let's write a script that backs up a specific log file.

1. Create a new script file: `touch log_backup.sh`
2. Make it executable: `chmod +x log_backup.sh`
3. Open it and add the shebang: `#!/bin/bash`
4. Add comments explaining the script's purpose.
5. Create a variable for the backup destination: `BACKUP_DIR="/home/your_user/backups"` (replace your_user).
6. Create a variable for the date: `DATE=$(date +%Y-%m-%d)`
7. Check if the backup directory exists. If not, create it.
`if [ ! -d "$BACKUP_DIR" ]; then mkdir -p "$BACKUP_DIR"; fi`
8. Define the source file to back up: `SOURCE_FILE="/var/log/syslog"`
9. Use the `cp` command to copy the file, adding the date to the filename.
`cp "$SOURCE_FILE" "$BACKUP_DIR/syslog-backup-$DATE.log"`
10. Add an echo statement to confirm that the backup was successful.
`echo "Backup of $SOURCE_FILE completed successfully to $BACKUP_DIR"`

The Idea Behind the Assignment

Why is scripting the superpower of a Linux systems administrator?

- **Consistency & Reliability:** A script performs a task the exact same way every single time, eliminating human error. When provisioning a dozen identical IoT sensors, a script ensures they are all configured perfectly.
 - **Efficiency:** A task that takes a human 10 minutes to do manually (like creating a user, setting a password, creating directories, and setting permissions) can be done by a script in less than a second.
 - **Scheduled Automation:** Using a tool called cron (which we'll hint at), you can make your scripts run automatically. For example, run a backup script every night at 2 AM, or run a health check script every hour and log the results. This is essential for maintaining intelligent systems that must run 24/7 without human intervention.
-

Assignment 5: Automated User Setup Script (12%)

Objective: To write a script that automates the process of creating a new user and a backup directory for them, including basic error checking.

Tasks:

1. Create a script named `create_project_user.sh`.
2. The script should prompt the user to enter a username using `read`.
3. Use an `if` statement to check if the user already exists.
 - **Hint:** You can use the command `id -u <username>`. This command will succeed (exit code 0) if the user exists and fail (non-zero exit code) if they don't. Check the exit code with `if [ $? -eq 0 ]`.
4. If the user already exists, the script should print an error message (e.g., "Error: User username already exists.") and `exit`.
5. If the user does not exist, the script should:
 - Create the user with a home directory (`sudo useradd -m ...`).
 - Create a directory for them inside `/srv/` named `project-<username>`.
 - Change the ownership of that directory to the new user.

- Print a success message like "Successfully created user username and their project directory."
-

Assignment Submission Instructions

- **File Name:** Name your submission file exactly as follows:
studentnum-studentname-Assignment5.pdf
 - *Example:* 13012070xx-MohammedEmadAlhams-Assignment5.pdf
 - **Terminal Prompt:** Your name must appear in the terminal prompt for all screenshots.
 - **File Format:** Your submission must be a **single PDF file** containing:
 - The complete code of your create_project_user.sh script (copy-paste as text).
 - A screenshot showing the script running successfully for a new user.
 - A second screenshot showing the script failing gracefully when you try to create the same user again.
 - A brief report (1-2 paragraphs) explaining how your if statement logic works to check if a user exists, specifically explaining the purpose of \$?.
 - **Academic Integrity:**
 - **Warning:** Identical submissions will be considered cheating and will result in a grade of **ZERO**.
 - **Warning:** AI-generated solutions or reports will result in a grade of **ZERO**.
-

Next Steps & Resources

Practice This Week:

- Modify the scripts from this lecture. Can you make the backup script use a for loop to back up multiple files?
- Write a simple script that asks for a number and tells the user if it's greater than 100.

Next Week: Lecture 7 - Advanced Bash Scripting

- We will make our scripts more powerful and reusable by learning about **functions**, handling lists of data with **arrays**, and implementing better **error handling**.

Gaza Connectivity Tips:

- Download this lecture PDF and save it offline.

- Write and test your script entirely offline. You only need to connect to submit the final PDF report.

Resources:

- Bash Scripting Cheat Sheet: <https://devhints.io/bash>
- Learn Shell - Interactive Tutorial: <https://www.learnshell.org/>
- Read the manual for bash: `man bash`
- Course Materials on Moodle